

SIMATS ENGINEERING  
ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

**COURSE OUTCOME COVERED**

*CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)*

*Assessment Tool Weightage: 20%*

QUESTIONS:

Total Marks:30

Annexure A

SHORT ANSWERS TEST

*Code of Conduct:*

*I Hemalatha.C (192524475) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

*Signature of the student: Hemalatha.C*

**Q1. Remove left recursion**

Given:

$$E \rightarrow E + T \mid T$$

The grammar is left recursive because  $E \rightarrow E + T$ .

After removing left recursion:

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \varepsilon$$

Answer:  $E \rightarrow T E'$  and  $E' \rightarrow + T E' \mid \varepsilon$ .

**Q2. Find FIRST(E)**

Given:

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \varepsilon$$

Since E starts with T:

$$\text{FIRST}(E) = \text{FIRST}(T)$$

Assuming  $T \rightarrow \text{id}$ :

$$\text{FIRST}(T) = \{ \text{id} \}$$

Answer:  $\text{FIRST}(E) = \{ \text{id} \}$ .

**Q3. Check whether the grammar is ambiguous**

Given:

$$E \rightarrow T + E \mid T$$

For  $T + T + T$ , there is only one structure:

$$T + (T + T)$$

Answer: The grammar is unambiguous.

**Q4. Check whether aba is valid**

Given:

$$S \rightarrow aSa \mid b$$

Derivation:

S

$\rightarrow aSa$

$\rightarrow aba$

Answer: Yes, aba is valid.

**Q5. Check whether ab is generated**

Given:

$S \rightarrow aA$

$A \rightarrow b$

Derivation:

S

$\rightarrow aA$

$\rightarrow ab$

Answer: Yes, ab is generated by the grammar.

**Q6. Construct parse tree for  $a + b * c$**

Given:

$E \rightarrow E + E \mid E * E \mid id$

Since  $*$  has higher precedence than  $+$ :

$a + (b * c)$

Parse tree:

E

$/ \mid \backslash$

E + E

$/ \backslash$

id E \* E

$|||$

a id id

||

b c

Result:  $a + (b * c)$ .

### **Q7. Apply left factoring**

Given:

$S \rightarrow aA \mid aB \mid b$

The common prefix is

a. After left factoring:

$S \rightarrow aS' \mid$

$b \mid S' \rightarrow A \mid$

B Answer:

$S \rightarrow aS' \mid$

$b \mid S' \rightarrow A \mid$

B

### **Q8. Find FOLLOW(A)**

Given:

$S \rightarrow AB$

$B \rightarrow b$

A is followed by B.

$\text{FOLLOW}(A) = \text{FIRST}(B)$

Since  $B \rightarrow b$ :

$\text{FIRST}(B) = \{ b \}$

Answer:  $\text{FOLLOW}(A) = \{ b \}$ .

### **Q9. Check whether aaabbb is valid**

Given:

$S \rightarrow aSb \mid \epsilon$

Derivation:

S

$\rightarrow aSb$

$\rightarrow aaSbb$

$\rightarrow aaaSbbb$

$\rightarrow aaabbb$

Answer: Yes, aaabbb is valid.

### **Q10. Convert ambiguous grammar to unambiguous grammar**

Given:

$E \rightarrow E + E \mid E * E \mid id$

Unambiguous grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

Here, \* has higher precedence than +, and both operators are left associative.

### **Q11. Is the grammar suitable for top-down parsing?**

Given:

$S \rightarrow Sa \mid b$

The grammar contains left recursion:  $S \rightarrow Sa$ .

Top-down parsers cannot directly handle left-recursive grammars.

After removing left recursion:

$S \rightarrow bS'$

$S' \rightarrow aS' \mid \epsilon$

Answer: The given grammar is not directly suitable for top-down parsing because it contains left recursion.

## Q12. Recursive descent parser

Given:

$S \rightarrow aS \mid b$

C Program:

```
#include
<stdio.h> char
input[100]; int pos
= 0;
void S()
{
if (input[pos] == 'a')
{
pos++;
; S();
}
else if (input[pos] == 'b')
{
pos++;
}
else
{
printf("Error");
}
}
int main()
{
printf("Enter string: ");
```

```

scanf("%s", input);
S();
if (input[pos] == '\0')
printf("Valid
string"); else
printf("Invalid string");
return 0;
}

```

Result: The parser accepts strings containing zero or more a's followed by b, such as b, ab and aab.

### Q13. Operator precedence table

Given:

$E \rightarrow E + E \mid E * E \mid id$

Since  $* > +$ :

| Stack/Input | + | \* | id | \$ |

|-----|---|---|---|

| + | > | < | < | > |

| \* | > | > | < | > |

| id | > | > | — | > |

| \$ | < | < | < | = |

< = Shift

> = Reduce

= = Equal precedence

### Q14. What is panic mode error recovery?

Answer:

Panic mode is a simple error recovery technique in which the parser discards input symbols until it finds a synchronizing symbol such as ;, } or \$. Then parsing continues from that point.

Example:

```
int a = ;
```

```
int b =
```

```
10;
```

The parser skips erroneous input until a suitable symbol such as ; is found and then continues parsing.

### **Q15. Why is error recovery important?**

Error recovery is important because it allows the compiler to continue parsing after detecting an error.

Advantages:

1. Detects multiple errors.
2. Prevents stopping at the first error.
3. Gives better error messages.
4. Saves debugging time.
5. Continues processing the remaining program.

Answer: Error recovery helps the compiler detect and report multiple errors efficiently while continuing compilation